Microservices

Part 2: SAGA and Strangler Design Patterns

Introduction

- This video is a continuation of Part 1, which discussed the "Decompose by Medium Position Pattern".
- The video focuses on three important patterns:
    - **Strangler Pattern:** Used for **refactoring** a monolithic service into microservices.
    - **Saga Pattern:** Solves the problem of **distributed transactions** across multiple databases.
    - **CQRS (Command Query Responsibility Segregation):** Separates command (write) and query (read) operations for improved performance and scalability.

Strangler Pattern:

- **Purpose:** Gradually refactoring a monolithic application into microservices.
- **How it Works:**
    - A "controller" is introduced to handle requests.
    - Initially, the controller forwards all traffic to the monolithic application.
    - Gradually, specific functionalities are extracted into microservices, and the controller routes traffic to them.
    - As more functionalities are migrated, the controller forwards less traffic to the monolithic application, eventually strangling it.
- **Advantages:**
    - Minimizes disruption to existing services.
    - Allows for a gradual transition to microservices.
- **Example:**
    - Imagine a monolithic e-commerce website being refactored into microservices.
    - The controller initially directs all traffic to the monolithic website.
    - Gradually, functionalities like order placement, inventory management, and payment processing are moved to individual microservices.
    - The controller gradually routes more traffic to these microservices, eventually reducing the reliance on the monolithic application.

Data Management in Microservices:

- **Two Approaches:**
    - **Database for Each Individual Service:** Each microservice has its own dedicated database, promoting autonomy and isolation.
    - **Shared Database:** All microservices share a single database, simplifying data access but potentially leading to complexities.
- **Why Database per Service is Preferred:**
    - **Scalability:** Allows for independent scaling of individual services without impacting others.
    - **Isolation:** Changes in one service's database don't affect others.
    - **Technology Flexibility:** Services can choose different databases based on their specific needs.
- **Advantages of Shared Database**
    - Join Query
    - Transactional Property (ACID)
- **Drawbacks of Shared Database:**
    - **Performance Bottlenecks:** Increased contention and performance issues as more services access the same database.
    - **Complexity:** Managing dependencies and ensuring consistency across multiple services becomes difficult.
    - **Limited Scalability:** Scaling the entire database is necessary, even if only one service needs more resources.

SAGA Pattern:

- **Purpose:** Managing distributed transactions across multiple databases, ensuring data consistency even if some operations fail.
- **How it Works:**
  - A sequence of local transactions is executed within each participating microservice.
  - Each transaction updates the database and publishes an event.
  - Subsequent transactions listen to these events and continue the process.
  - In case of failure, compensation events are published to undo completed operations and maintain consistency.
- **Types of Sagas:**
  - **Choreography:** Each service manages its own transactions and listens to events from other services.
  - **Orchestration:** A centralized orchestrator manages the transaction flow and handles compensation logic.
- **Example:**
  - An order processing saga involving services for order creation, inventory management, and payment processing.
  - If the payment service fails, compensation events are triggered to cancel the order and update inventory.
- **Advantages:**
  - Guarantees data consistency in distributed systems.
  - Provides a mechanism for handling failures and rollbacks.
  - Allows for flexibility in service interactions.
- **Disadvantages:**
  - Increased complexity compared to local transactions.
  - Requires careful design and implementation to ensure correctness.
- **Interview Question Example:**
  - Explain how you would handle a transaction involving transferring money between two users in a microservice architecture.

CQRS Pattern (Command Query Request Segregation):

- **Purpose:** Separating read (query) operations from write (command) operations for better performance and scalability.
- **How it Works:**
  - The system maintains separate models for read and write operations.
  - Write operations are performed through commands, updating the write model.
  - Read operations access the read model, which can be optimized for fast retrieval.
- **Advantages:**
  - **Performance Improvement:** Optimized read models can handle queries more efficiently.
  - **Scalability:** Read and write models can be scaled independently based on their specific needs.
  - **Flexibility:** Allows for different data structures and query languages for read and write operations.
- **Example:**
  - A blog application where write operations are performed on a relational database, while read operations access a denormalized view optimized for fast search.
- **Challenges:**
  - Maintaining consistency between the read and write models.
  - Ensuring the read model is up-to-date with changes in the write model.